

Task Tracker API

Exercises 2-4: Complete Questions and Answers

Requirements • SQL Schema • Controllers • Security Audit • OpenAPI • Unit Tests •
Deliverables

Compiled from this ChatGPT conversation
15 September 2026

Exercise 2 - Generate Requirements, Schema, and Controllers

Question

Task: Use Least-to-Most prompting to generate software requirements and user stories, then design SQL schemas, and finally generate route controllers using the CRISPE framework.

Answer - Setup Steps:

- Least-to-Most Requirements Prompt: Act as a Business Analyst. We want to design a backend REST API named Task Tracker API. First, outline the system architecture scope and write 3 core B2B user stories (for User Signup, Task Creation, and Task Updates) with acceptance criteria. Do not write database schemas or backend code yet. Save the output to `srd_draft.txt` (REQ-001).
- SQL Database Schema Prompt: Generate relational SQL table schemas for users (id, name, email, password_hash, created_at) and tasks (id, user_id, title, description, status, due_date, created_at), including NOT NULL, UNIQUE, FOREIGN KEY, and PRIMARY KEY constraints, with raw DDL SQL output (REQ-003).
- CRISPE Coding Prompt: Act as a Senior Backend Software Engineer and write route controller logic in Node.js/Express or Python/FastAPI for POST `/api/v1/tasks` and GET `/api/v1/tasks/:id`, following DRY/SOLID principles, using raw SQL (no ORM), with inline comments (REQ-002, REQ-004). Save the code draft in `code_repository/`.

Answer

Stage 1 - Least-to-Most Requirements Prompt

Use this prompt first:

Act as a Business Analyst. We want to design a backend REST API named "Task Tracker API". First, outline the system architecture scope and write 3 core B2B user stories (for User Signup, Task Creation, and Task Updates) with acceptance criteria. Do not write database schemas or backend code yet.

Requirements output summary:

- User Signup - register with name, email, and password; enforce unique email and hashed-password storage.
- Task Creation - authenticated users can create tasks containing title, description, status, and optional due date.
- Task Updates - authorized users can update their existing task information with validation and ownership checks.

Stage 2 - SQL Database Schema Prompt

Based on the approved Task Tracker API requirements, act as a Database Engineer. Generate relational SQL table schemas for users and tasks.

Generated SQL schema:

```
CREATE TABLE users (  
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  name VARCHAR(150) NOT NULL,  
  email VARCHAR(255) NOT NULL UNIQUE,  
  password_hash VARCHAR(255) NOT NULL,  
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
```

```
);

CREATE TABLE tasks (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  user_id BIGINT NOT NULL,
  title VARCHAR(255) NOT NULL,
  description TEXT,
  status VARCHAR(30) NOT NULL DEFAULT 'pending',
  due_date DATE,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_tasks_user
    FOREIGN KEY (user_id)
    REFERENCES users(id)
    ON DELETE CASCADE,
  CONSTRAINT chk_tasks_status
    CHECK (status IN ('pending', 'in_progress', 'completed'))
);
```

Stage 3 - CRISPE Controller Prompt

Capacity: Act as a Senior Backend Software Engineer experienced with Node.js, Express, PostgreSQL, REST APIs, secure coding, and enterprise backend architecture.

Recipient: The code is intended for backend developers maintaining a B2B Task Tracker REST API.

Instruction: Implement controller logic for POST /api/v1/tasks and GET /api/v1/tasks/:id. Use raw parameterized SQL queries and do not use an ORM.

Style: Write clean, maintainable JavaScript following DRY and SOLID principles. Include meaningful inline comments, consistent JSON responses, and clear validation logic.

Parameters: Validate task IDs, authenticated user IDs, title, task status, and due dates. Restrict status to pending, in_progress, or completed. Prevent SQL injection with parameterized queries. Ensure users can retrieve only their own tasks. Return suitable HTTP status codes including 200, 201, 400, 401, 404, and 500.

Evaluation: The solution should be secure, readable, reusable, RESTful, and suitable as a production-oriented controller draft.

Generated task controller:

```
const db = require('./db');

const ALLOWED_STATUSES = new Set(['pending', 'in_progress', 'completed']);

function isPositiveInteger(value) {
  return Number.isInteger(Number(value)) && Number(value) > 0;
}

function isValidDate(value) {
  if (!value) return true;
  return !Number.isNaN(Date.parse(value));
}

/**
 * POST /api/v1/tasks
 * Creates a new task using parameterized raw SQL.
 * Assumption: authenticated middleware stores the current user ID in req.user.id.
 */
async function createTask(req, res) {
  const userId = req.user?.id;
  const { title, description = null, status = 'pending', due_date = null } = req.body;

  if (!isPositiveInteger(userId)) {
```

```

    return res.status(401).json({ error: 'Unauthorized' });
  }

  if (!title || typeof title !== 'string' || !title.trim()) {
    return res.status(400).json({ error: 'title is required' });
  }

  if (!ALLOWED_STATUSES.has(status)) {
    return res.status(400).json({ error: 'Invalid task status' });
  }

  if (!isValidDate(due_date)) {
    return res.status(400).json({ error: 'Invalid due_date' });
  }

  try {
    // Verify ownership target exists before inserting the task.
    const userResult = await db.query(
      'SELECT id FROM users WHERE id = $1',
      [userId]
    );

    if (userResult.rowCount === 0) {
      return res.status(404).json({ error: 'User not found' });
    }

    // Parameterized SQL protects the query from SQL injection.
    const taskResult = await db.query(
      `INSERT INTO tasks (user_id, title, description, status, due_date)
      VALUES ($1, $2, $3, $4, $5)
      RETURNING id, user_id, title, description, status, due_date, created_at`,
      [userId, title.trim(), description, status, due_date]
    );

    return res.status(201).json({ data: taskResult.rows[0] });
  } catch (error) {
    console.error('createTask error:', error);
    return res.status(500).json({ error: 'Internal server error' });
  }
}

/**
 * GET /api/v1/tasks/:id
 * Retrieves one task owned by the authenticated user.
 */
async function getTaskById(req, res) {
  const userId = req.user?.id;
  const taskId = req.params.id;

  if (!isPositiveInteger(userId)) {
    return res.status(401).json({ error: 'Unauthorized' });
  }

  if (!isPositiveInteger(taskId)) {
    return res.status(400).json({ error: 'Invalid task ID' });
  }

  try {
    const result = await db.query(
      `SELECT id, user_id, title, description, status, due_date, created_at
      FROM tasks
      WHERE id = $1 AND user_id = $2`,
      [taskId, userId]
    );

    if (result.rowCount === 0) {
      return res.status(404).json({ error: 'Task not found' });
    }
  }
}

```

```

    return res.status(200).json({ data: result.rows[0] });
  } catch (error) {
    console.error('getTaskById error:', error);
    return res.status(500).json({ error: 'Internal server error' });
  }
}

module.exports = {
  createTask,
  getTaskById,
};

```

Generated routes:

```

const express = require('express');
const { createTask, getTaskById } = require('./tasks.controller');

const router = express.Router();

router.post('/api/v1/tasks', createTask);
router.get('/api/v1/tasks/:id', getTaskById);

module.exports = router;

```

Database connection module:

```

const { Pool } = require('pg');

const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
});

module.exports = {
  query: (text, params) => pool.query(text, params),
};

```

Exercise 2 result: REQ-001, REQ-002, REQ-003, and REQ-004 completed.

Exercise 3 - Run Secure Code Reflection and Debugging

Question

Task: Audit your generated controllers for security exploits (such as SQL injection) using a reflection prompt, and verify the refactored output uses parameterized queries.

Setup Steps:

- Copy a mock vulnerable endpoint (e.g., a `/api/v1/login` route built with string-concatenated SQL) into ChatGPT as the code to audit.
- Secure Reflection Prompt: Act as a Security Auditor. Scan the code block below for security vulnerabilities (e.g., SQL Injection, unhandled crashes). Identify the issue, explain the risk, and write a refactored, secure version of this endpoint using parameterized SQL queries and try/catch blocks (REQ-005, TR-003).
- Verify the output replaces the vulnerable query with a parameterized query.

Answer

Step 1 - Mock Vulnerable Endpoint

```
app.post('/api/v1/login', async (req, res) => {
  const { email, password } = req.body;
  const query =
    "SELECT * FROM users WHERE email = '" + email +
    "' AND password = '" + password + "'";
  const result = await db.query(query);
  if (result.rows.length > 0) {
    res.status(200).json({ message: 'Login successful', user: result.rows[0] });
  } else {
    res.status(401).json({ message: 'Invalid credentials' });
  }
});
```

Step 2 - Secure Reflection Prompt

Act as a Security Auditor. Scan the code block below for security vulnerabilities such as SQL Injection, unhandled crashes

Security Audit Findings

- SQL Injection - Critical: string-concatenated SQL allows attacker-controlled input to alter the query.
- Plain-text password comparison - Critical: passwords should be stored as secure hashes and checked with bcrypt or Argon2.
- No error handling: an unhandled database/runtime failure can destabilize the endpoint and may expose internal errors.
- Missing input validation: email and password should be checked before database access.

Step 3 - Secure Refactored Endpoint

```
const bcrypt = require('bcrypt');

app.post('/api/v1/login', async (req, res) => {
  try {
    const { email, password } = req.body;
    if (!email || !password) {
      return res.status(400).json({ message: 'Email and password are required' });
    }
  }
});
```

```

const query = `
  SELECT id, name, email, password_hash
  FROM users
  WHERE email = $1
`;
const result = await db.query(query, [email]);

if (result.rows.length === 0) {
  return res.status(401).json({ message: 'Invalid email or password' });
}

const user = result.rows[0];
const passwordMatches = await bcrypt.compare(password, user.password_hash);
if (!passwordMatches) {
  return res.status(401).json({ message: 'Invalid email or password' });
}

return res.status(200).json({
  message: 'Login successful',
  user: { id: user.id, name: user.name, email: user.email }
});
} catch (error) {
  console.error('Login error:', error);
  return res.status(500).json({ message: 'Internal server error' });
}
});

```

Step 4 - Parameterized Query Verification

```

const query = `SELECT id, name, email, password_hash FROM users WHERE email = $1`;
const result = await db.query(query, [email]);

```

Verification: user input is passed separately from SQL text, so it is treated as data rather than executable SQL. For MySQL, the equivalent placeholder is ? with `db.query(query, [email])`.

Exercise 3 result: REQ-005 passed and TR-003 passed.

Exercise 4 - Generate Documentation, Tests, and Compile Deliverables

Question

Task: Generate Swagger/OpenAPI documentation and unit tests for your endpoints, then compile all outputs into structured Markdown deliverable files.

Setup Steps:

- Generate Swagger/OpenAPI YAML documentation for POST /api/v1/tasks and GET /api/v1/tasks/:id, including request schema, 200 success response, and 400 error response blocks (REQ-006).
- Act as a QA Automation Engineer and write 2 unit tests in Jest/Supertest validating successful task creation (status 201) and getting a task with an invalid ID (status 404) (REQ-006).
- Create four files (TR-001): api_documentation.md, test_cases.md, prompt_library.md, and software_requirement_document.md.

Answer

api_documentation.md

Swagger/OpenAPI 3.0 documentation for the task endpoints.

```
# Task Tracker API – API Documentation

## Requirement Coverage

- **REQ-006** – Swagger/OpenAPI documentation for task creation and task retrieval.

## OpenAPI Specification

```yaml
openapi: 3.0.3
info:
 title: Task Tracker API
 version: 1.0.0
 description: REST API for creating and retrieving user-owned tasks.

servers:
 - url: http://localhost:3000
 description: Local development server

paths:
 /api/v1/tasks:
 post:
 summary: Create a new task
 operationId: createTask
 tags:
 - Tasks
 requestBody:
 required: true
 content:
 application/json:
 schema:
 $ref: '#/components/schemas/CreateTaskRequest'
 example:
 title: Prepare monthly report
 description: Compile the September operations report
 status: pending
 due_date: '2026-09-30'
 responses:
 '201':
 description: Task created successfully
```

```

content:
application/json:
schema:
$ref: '#/components/schemas/TaskResponse'
example:
message: Task created successfully
task:
id: 101
user_id: 12
title: Prepare monthly report
description: Compile the September operations report
status: pending
due_date: '2026-09-30'
created_at: '2026-09-15T12:00:00Z'
'400':
description: Invalid request data
content:
application/json:
schema:
$ref: '#/components/schemas/ErrorResponse'
example:
message: Title is required
'401':
description: Authentication required
content:
application/json:
schema:
$ref: '#/components/schemas/ErrorResponse'
'500':
description: Internal server error
content:
application/json:
schema:
$ref: '#/components/schemas/ErrorResponse'

/api/v1/tasks/{id}:
get:
summary: Get a task by ID
operationId: getTaskById
tags:
- Tasks
parameters:
- name: id
in: path
required: true
description: Numeric task identifier
schema:
type: integer
minimum: 1
responses:
'200':
description: Task retrieved successfully
content:
application/json:
schema:
$ref: '#/components/schemas/TaskResponse'
example:
message: Task retrieved successfully
task:
id: 101
user_id: 12
title: Prepare monthly report
description: Compile the September operations report
status: pending
due_date: '2026-09-30'
created_at: '2026-09-15T12:00:00Z'
'400':
description: Invalid task ID
content:
application/json:
schema:
$ref: '#/components/schemas/ErrorResponse'
example:
message: Invalid task ID

```

```

'404':
description: Task not found
content:
application/json:
schema:
$ref: '#/components/schemas/ErrorResponse'
example:
message: Task not found
'500':
description: Internal server error
content:
application/json:
schema:
$ref: '#/components/schemas/ErrorResponse'

```

```

components:
schemas:
CreateTaskRequest:
type: object
required:
- title
properties:
title:
type: string
minLength: 1
maxLength: 255
description:
type: string
nullable: true
status:
type: string
enum:
- pending
- in_progress
- completed
default: pending
due_date:
type: string
format: date
nullable: true

```

```

Task:
type: object
required:
- id
- user_id
- title
- status
- created_at
properties:
id:
type: integer
user_id:
type: integer
title:
type: string
description:
type: string
nullable: true
status:
type: string
enum:
- pending
- in_progress
- completed
due_date:
type: string
format: date
nullable: true
created_at:
type: string
format: date-time

```

```
TaskResponse:
```

```

type: object
properties:
 message:
 type: string
 task:
 $ref: '#/components/schemas/Task'

```

```

ErrorResponse:
 type: object
 required:
 - message
 properties:
 message:
 type: string
 ``

```

## ## Verification Notes

- `POST /api/v1/tasks` documents request validation and a `201 Created` success response.
- `GET /api/v1/tasks/{id}` documents `200`, `400`, and `404` responses.
- Both endpoints include structured JSON error responses.

## test\_cases.md

Jest/Supertest unit tests plus the QA test matrix.

```
Task Tracker API – Unit Tests and QA Matrix
```

```
Requirement Coverage
```

- **REQ-006** – Automated tests for task creation and invalid/nonexistent task retrieval.

```
Jest / Supertest Unit Tests
```

The examples below assume the Express application is exported from `app.js` and the database module is exported from `db.js`.

```

```javascript
const request = require('supertest');
const app = require('./app');
const db = require('./db');

jest.mock('./db');

describe('Task Tracker API', () => {
  afterEach(() => {
    jest.clearAllMocks();
  });

  test('POST /api/v1/tasks should return 201 when task creation succeeds', async () => {
    // Mock authenticated user lookup.
    db.query
      .mockResolvedValueOnce({ rows: [{ id: 12 }] })
    // Mock task insertion result.
    db.insert
      .mockResolvedValueOnce({
        rows: [
          {
            id: 101,
            user_id: 12,
            title: 'Prepare monthly report',
            description: 'Compile the September operations report',
            status: 'pending',
            due_date: '2026-09-30',
            created_at: '2026-09-15T12:00:00Z'
          }
        ]
      });

    const response = await request(app)
      .post('/api/v1/tasks')
      // For test purposes, assume authentication middleware reads x-user-id.
      .set('x-user-id', '12')
      .send({

```

```

    title: 'Prepare monthly report',
    description: 'Compile the September operations report',
    status: 'pending',
    due_date: '2026-09-30'
  });

  expect(response.status).toBe(201);
  expect(response.body.message).toBe('Task created successfully');
  expect(response.body.task).toMatchObject({
    id: 101,
    user_id: 12,
    title: 'Prepare monthly report',
    status: 'pending'
  });
});

test('GET /api/v1/tasks/:id should return 404 when the task does not exist', async () => {
  // Valid numeric ID, but no matching task belongs to the authenticated user.
  db.query.mockResolvedValueOnce({ rows: [] });

  const response = await request(app)
    .get('/api/v1/tasks/99999')
    .set('x-user-id', '12');

  expect(response.status).toBe(404);
  expect(response.body).toEqual({
    message: 'Task not found'
  });
});
`);

```

> Note: If the implementation treats a malformed value such as `abc` as an invalid ID, that case should normally return `400`. The required `404` test above therefore uses a syntactically valid task ID that does not exist.

QA Test Matrix

Test ID	Endpoint	Scenario	Input / Preconditions	Expected Result	Expected Status
TC-001	POST `/api/v1/tasks`	Successful task creation	Authenticated user; valid title, description, status, due date	Task is created and returned	201
TC-002	POST `/api/v1/tasks`	Missing title	Authenticated user; title omitted	Validation error returned	400
TC-003	POST `/api/v1/tasks`	Invalid status	Status outside `pending`, `in_progress`, `completed`	Validation error returned	400
TC-004	POST `/api/v1/tasks`	Unauthenticated request	No authenticated user context	Authentication error returned	401
TC-005	GET `/api/v1/tasks/:id`	Existing owned task	Valid task ID owned by current user	Task is returned	200
TC-006	GET `/api/v1/tasks/:id`	Nonexistent task	Valid numeric ID with no matching record	Not-found response returned	404
TC-007	GET `/api/v1/tasks/:id`	Malformed task ID	`id=abc`	Validation error returned	400
TC-008	GET `/api/v1/tasks/:id`	Task belongs to another user	Valid ID but ownership query returns no row	Task is not disclosed	404
TC-009	Both	Database failure	Mock DB throws exception	Generic server error returned; process does not crash	500
TC-010	Both	SQL injection attempt	Malicious text submitted in string fields	Input is treated as data through parameterized SQL	No SQL injection

prompt_library.md

Reusable Least-to-Most, SQL, CRISPE, Reflection, API documentation, and unit-test prompts.

Task Tracker API – Prompt Library

This file contains reusable prompt templates used across Exercises 2-4.

1. Least-to-Most Requirements Prompt Template

```text

Act as a Business Analyst.

We want to design a backend REST API named "{API\_NAME}".

First, outline the system architecture scope and write {{USER\_STORY\_COUNT}} core B2B user stories for: {{USER\_STORY\_TOPICS}}.

For each user story, include clear acceptance criteria.

Do not write database schemas or backend code yet.

Output should be suitable for saving as {{REQUIREMENTS\_FILENAME}}.

```

Example Parameter Values

```
```text
{{API_NAME}} = Task Tracker API
{{USER_STORY_COUNT}} = 3
{{USER_STORY_TOPICS}} = User Signup, Task Creation, Task Updates
{{REQUIREMENTS_FILENAME}} = srd_draft.txt
```
```

2. SQL Schema Prompt Template

```
```text
Act as a Database Engineer.

Using the approved requirements for {{API_NAME}}, generate relational SQL table schemas for:
{{TABLE_DEFINITIONS}}.
```

Include the following constraints where appropriate:

- PRIMARY KEY
- FOREIGN KEY
- NOT NULL
- UNIQUE

Use {{DATABASE\_ENGINE}} syntax.

Return raw DDL SQL only.

Do not use an ORM.

```

Example Parameter Values

```
```text
{{API_NAME}} = Task Tracker API
{{TABLE_DEFINITIONS}} = users (id, name, email, password_hash, created_at) and tasks (id, user_id, title,
description, status, due_date, created_at)
{{DATABASE_ENGINE}} = PostgreSQL
```
```

3. CRISPE Coding Prompt Template

```
```text
Capacity:
Act as a Senior Backend Software Engineer experienced with {{FRAMEWORK}}, {{DATABASE_ENGINE}}, REST APIs, secure
coding, and enterprise backend architecture.
```

Recipient:

The code is intended for {{TARGET\_RECIPIENT}} maintaining {{API\_NAME}}.

Instruction:

Write route controller logic for the following endpoints:

{{ENDPOINTS}}.

Use raw parameterized SQL queries and do not use an ORM.

Style:

Write clean, maintainable {{PROGRAMMING\_LANGUAGE}} code following DRY and SOLID principles. Include meaningful inline comments, consistent JSON responses, and clear validation logic.

Parameters:

- Validate {{VALIDATION\_RULES}}.
- Prevent SQL injection using parameterized queries.
- Enforce authorization/ownership rules where applicable.
- Return appropriate HTTP status codes including {{STATUS\_CODES}}.
- Use try/catch error handling.

Evaluation:

The solution must be secure, readable, reusable, RESTful, testable, and suitable as a production-oriented controller draft.

### ### Example Parameter Values

```
```text
{{FRAMEWORK}} = Node.js/Express
{{DATABASE_ENGINE}} = PostgreSQL
{{TARGET_RECIPIENT}} = backend developers
{{API_NAME}} = Task Tracker API
{{ENDPOINTS}} = POST /api/v1/tasks and GET /api/v1/tasks/:id
{{PROGRAMMING_LANGUAGE}} = JavaScript
{{VALIDATION_RULES}} = authenticated user ID, task ID, title, status, and due date
{{STATUS_CODES}} = 200, 201, 400, 401, 404, and 500
```
```

### ## 4. Secure Reflection Prompt Template

```
```text
Act as a Security Auditor.

Scan the code block below for security vulnerabilities such as:
- SQL Injection
- unhandled crashes
- weak credential handling
- unsafe input handling
- authorization failures
```

Identify each issue, explain the associated risk, and write a refactored secure version of the endpoint.

Requirements for the refactored version:

- Use parameterized SQL queries.
- Use try/catch blocks.
- Validate required inputs.
- Return appropriate HTTP status codes.
- Avoid disclosing sensitive implementation details.
- Follow secure backend development practices.

Technology stack: {{FRAMEWORK}} with {{DATABASE_ENGINE}}.

Code to audit:

```
{{CODE_BLOCK}}
```
```

### ### Example Parameter Values

```
```text
{{FRAMEWORK}} = Node.js/Express
{{DATABASE_ENGINE}} = PostgreSQL
{{CODE_BLOCK}} = vulnerable endpoint source code
```
```

### ## 5. API Documentation Prompt Template

```
```text
Act as an API Documentation Engineer.

Generate Swagger/OpenAPI {{OPENAPI_VERSION}} YAML documentation for:
{{ENDPOINTS}}.
```

Include:

- request schemas
- path parameters
- success responses
- 400 error responses
- 404 responses where applicable
- reusable component schemas
- JSON examples

API name: {{API_NAME}}.

Return the result as a valid YAML block suitable for embedding in Markdown.

```
```
```

```
6. Unit Test Prompt Template

```text
Act as a QA Automation Engineer.

Write {{TEST_COUNT}} automated unit/integration tests using {{TEST_FRAMEWORK}} for {{API_NAME}}.

Required scenarios:
{{TEST_SCENARIOS}}.

Mock database calls where necessary.
Assert both HTTP status codes and important response body fields.
Keep the tests independent and deterministic.
```
```

## software\_requirement\_document.md

Consolidated architecture scope, user stories, SQL scripts, controllers, routes, and traceability.

```
Software Requirement Document – Task Tracker API

1. Purpose

The Task Tracker API is a backend REST service that allows business users to register, create tasks, update tasks, and retrieve task information securely. The initial implementation uses Node.js/Express and PostgreSQL with raw parameterized SQL queries.

2. Architecture Scope

The system consists of:

- REST API layer built with Node.js and Express.
- PostgreSQL relational database.
- Authentication middleware that provides the authenticated user identity to controllers.
- Controller/service logic for task operations.
- Raw SQL using parameterized queries; no ORM.
- JSON request and response payloads.
- Validation, authorization, and centralized-safe error handling.

Out of Scope for This Draft

- Frontend user interface.
- OAuth/SSO integration.
- Notifications and reminders.
- File attachments.
- Advanced reporting.

3. Core User Stories

US-001 – User Signup

As a business user,
I want to register for the Task Tracker API,
so that I can securely access task management functionality.

Acceptance Criteria

1. User provides name, email, and password.
2. Email must be unique.
3. Required fields cannot be empty.
4. Password must be stored as a secure hash, never plain text.
5. Successful registration returns an appropriate success status.
6. Duplicate email registration is rejected.

US-002 – Task Creation

As an authenticated business user,
I want to create a task,
so that I can track work that must be completed.

Acceptance Criteria
```

1. Only authenticated users can create tasks.
2. A task must contain a non-empty title.
3. Description and due date may be optional.
4. Status must be one of `pending`, `in\_progress`, or `completed`.
5. The new task is associated with the authenticated user's ID.
6. Successful creation returns HTTP `201` and the created task.

### ### US-003 – Task Updates

**\*\*As an\*\*** authenticated business user,  
**\*\*I want\*\*** to update my task,  
**\*\*so that\*\*** its current details and progress remain accurate.

### #### Acceptance Criteria

1. Only the owning user may update a task.
2. Invalid task IDs are rejected.
3. Updated status must remain within the allowed values.
4. Changes are saved to the correct task record.
5. Attempts to update inaccessible or nonexistent tasks do not expose another user's data.
6. Database errors return a safe server-error response.

## ## 4. Relational Database Schema

```
```sql
CREATE TABLE users (
  id BIGSERIAL PRIMARY KEY,
  name VARCHAR(150) NOT NULL,
  email VARCHAR(255) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE tasks (
  id BIGSERIAL PRIMARY KEY,
  user_id BIGINT NOT NULL,
  title VARCHAR(255) NOT NULL,
  description TEXT,
  status VARCHAR(30) NOT NULL DEFAULT 'pending',
  due_date DATE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

  CONSTRAINT fk_tasks_user
  FOREIGN KEY (user_id)
  REFERENCES users(id)
  ON DELETE CASCADE,

  CONSTRAINT chk_tasks_status
  CHECK (status IN ('pending', 'in_progress', 'completed'))
);

CREATE INDEX idx_tasks_user_id ON tasks(user_id);
```
```

## ## 5. Final Task Controllers

### ### `tasks.controller.js`

```
```javascript
const db = require('./db');

const ALLOWED_STATUSES = new Set([
  'pending',
  'in_progress',
  'completed'
]);

/**
 * Creates a task for the authenticated user.
 */
async function createTask(req, res) {
  try {
    const userId = Number(req.user?.id || req.headers['x-user-id']);
    const {

```

```

    title,
    description = null,
    status = 'pending',
    due_date = null
  } = req.body;

  if (!Number.isInteger(userId) || userId <= 0) {
    return res.status(401).json({
      message: 'Authentication required'
    });
  }

  if (!title || typeof title !== 'string' || !title.trim()) {
    return res.status(400).json({
      message: 'Title is required'
    });
  }

  if (!ALLOWED_STATUSES.has(status)) {
    return res.status(400).json({
      message: 'Invalid task status'
    });
  }

  if (due_date && Number.isNaN(Date.parse(due_date))) {
    return res.status(400).json({
      message: 'Invalid due date'
    });
  }

  // Confirm that the authenticated user exists.
  const userResult = await db.query(
    'SELECT id FROM users WHERE id = $1',
    [userId]
  );

  if (userResult.rows.length === 0) {
    return res.status(401).json({
      message: 'Invalid authenticated user'
    });
  }

  // Parameter placeholders keep values separate from executable SQL.
  const insertQuery = `
  INSERT INTO tasks
  (user_id, title, description, status, due_date)
  VALUES
  ($1, $2, $3, $4, $5)
  RETURNING
  id,
  user_id,
  title,
  description,
  status,
  due_date,
  created_at
  `;

  const values = [
    userId,
    title.trim(),
    description,
    status,
    due_date
  ];

  const result = await db.query(insertQuery, values);

  return res.status(201).json({
    message: 'Task created successfully',
    task: result.rows[0]
  });
} catch (error) {
  console.error('Create task error:', error);
}

```

```

    return res.status(500).json({
      message: 'Internal server error'
    });
  }
}

/**
 * Retrieves one task owned by the authenticated user.
 */
async function getTaskById(req, res) {
  try {
    const userId = Number(req.user?.id || req.headers['x-user-id']);
    const taskId = Number(req.params.id);

    if (!Number.isInteger(userId) || userId <= 0) {
      return res.status(401).json({
        message: 'Authentication required'
      });
    }

    if (!Number.isInteger(taskId) || taskId <= 0) {
      return res.status(400).json({
        message: 'Invalid task ID'
      });
    }

    // Ownership is enforced in the SQL WHERE clause.
    const query = `
    SELECT
      id,
      user_id,
      title,
      description,
      status,
      due_date,
      created_at
    FROM tasks
    WHERE id = $1
    AND user_id = $2
    `;

    const result = await db.query(query, [taskId, userId]);

    if (result.rows.length === 0) {
      return res.status(404).json({
        message: 'Task not found'
      });
    }

    return res.status(200).json({
      message: 'Task retrieved successfully',
      task: result.rows[0]
    });
  } catch (error) {
    console.error('Get task error:', error);

    return res.status(500).json({
      message: 'Internal server error'
    });
  }
}

module.exports = {
  createTask,
  getTaskById
};
``

## 6. Express Routes

``javascript
const express = require('express');
const {

```

```

    createTask,
    getTaskById
  } = require('./tasks.controller');

  const router = express.Router();

  router.post('/api/v1/tasks', createTask);
  router.get('/api/v1/tasks/:id', getTaskById);

  module.exports = router;
}

```

7. Secure Login Reflection Example

A vulnerable login implementation that concatenates user input directly into SQL must be rejected because it can enable SQL injection.

Secure Pattern

```

```javascript
const bcrypt = require('bcrypt');

app.post('/api/v1/login', async (req, res) => {
 try {
 const { email, password } = req.body;

 if (!email || !password) {
 return res.status(400).json({
 message: 'Email and password are required'
 });
 }

 const query = `
 SELECT id, name, email, password_hash
 FROM users
 WHERE email = $1
 `;

 const result = await db.query(query, [email]);

 if (result.rows.length === 0) {
 return res.status(401).json({
 message: 'Invalid email or password'
 });
 }

 const user = result.rows[0];
 const passwordMatches = await bcrypt.compare(
 password,
 user.password_hash
);

 if (!passwordMatches) {
 return res.status(401).json({
 message: 'Invalid email or password'
 });
 }

 return res.status(200).json({
 message: 'Login successful',
 user: {
 id: user.id,
 name: user.name,
 email: user.email
 }
 });
 } catch (error) {
 console.error('Login error:', error);

 return res.status(500).json({
 message: 'Internal server error'
 });
 }
});

```

```

...

8. Security Verification

- SQL values use PostgreSQL placeholders such as `$1` and `$2`.
- User-supplied values are passed separately in parameter arrays.
- No controller builds SQL using string concatenation of request data.
- `try/catch` prevents unhandled asynchronous database failures.
- Ownership filtering prevents one user from retrieving another user's task.
- Password verification uses a hash comparison rather than a plaintext database password.

9. Requirement Traceability

| Requirement | Deliverable / Implementation |
|---|---|
| REQ-001 | Architecture scope and B2B user stories |
| REQ-002 | Node.js/Express task controllers |
| REQ-003 | PostgreSQL relational schema |
| REQ-004 | DRY/SOLID, raw SQL, parameterized queries, inline comments |
| REQ-005 | Security reflection and secure login refactoring |
| REQ-006 | OpenAPI documentation and Jest/Supertest tests |
| TR-001 | Structured Markdown deliverable files |
| TR-003 | Reflection/debugging and secure implementation verification |

```

## Important API validation note

A malformed task ID such as abc should return 400 Bad Request. A syntactically valid numeric task ID that does not exist should return 404 Not Found. Therefore, the required 404 test uses a nonexistent numeric task ID.

Exercise 4 result: REQ-006 and TR-001 completed.